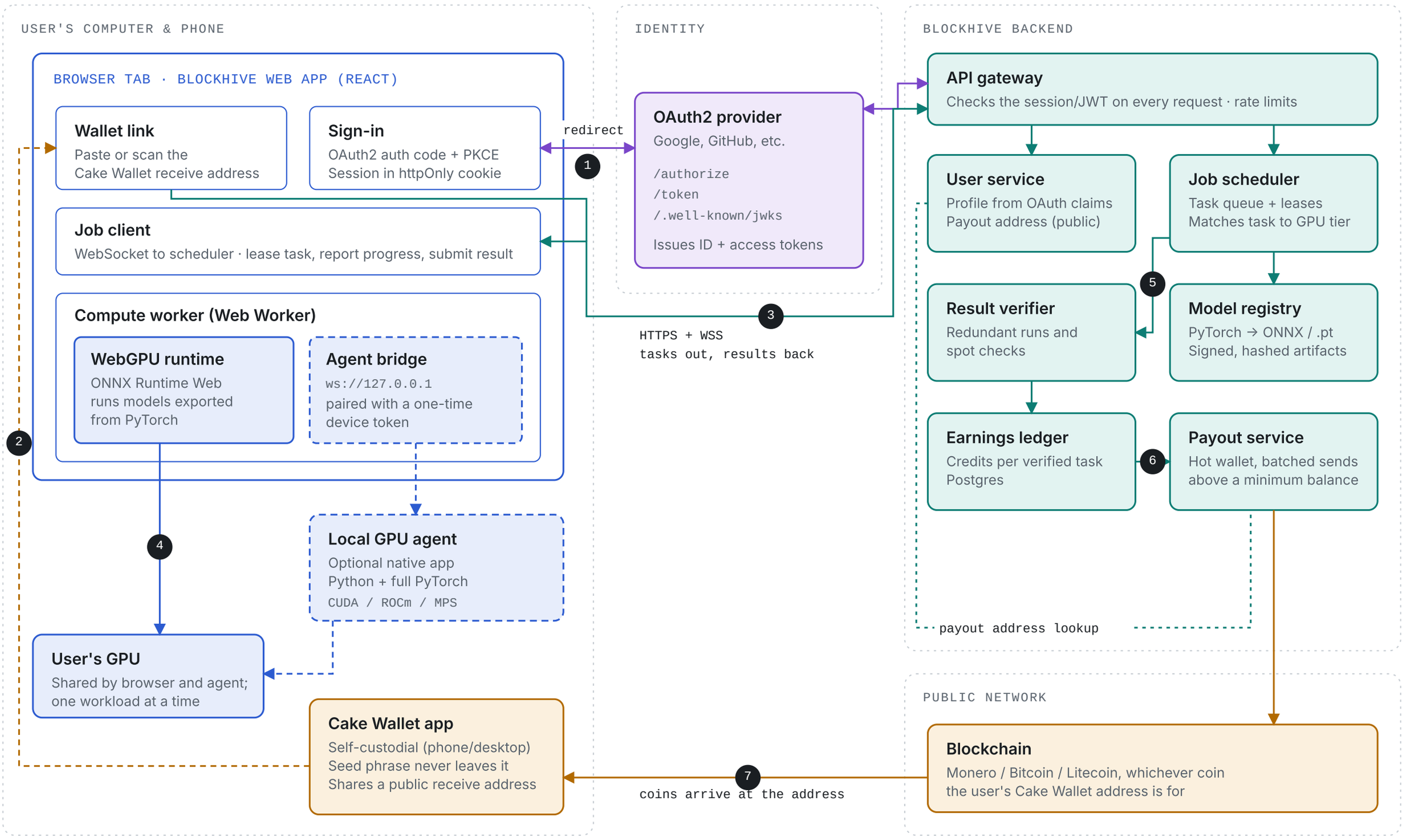
BLOCKHIVE · SYSTEM ARCHITECTURE

Browser GPU compute with OAuth2 sign-in and Cake Wallet payouts

How a user signs in, links a self-custodial Cake Wallet address, runs PyTorch workloads on their own GPU from a browser tab, and gets paid for verified work.

User's computer · OAuth2 provider · Blockhive backend · Wallet & blockchain · Optional component



Request flow

1. **Sign in with OAuth2.**
   The tab redirects to the provider using the authorization code flow with PKCE. The provider returns a code, and the backend swaps it for tokens server-side, then sets an httpOnly session cookie. No client secret or token ever sits in the JavaScript bundle.

2. **Link a Cake Wallet address.**
   The user creates a wallet inside the Cake Wallet app, then copies or scans a receive address into the tab. The backend checks the address format for that coin and stores only the public address.

3. **Lease a task.**
   The job client opens a WebSocket to the scheduler, reports its GPU tier, and leases a task: model ID, input shard and deadline. The model is downloaded from the registry and its hash is checked before it runs.

4. **Run it on the GPU.**
   By default the Web Worker runs the PyTorch-exported model through WebGPU. If the user installed the local agent, the tab hands the task to it over `127.0.0.1` and full PyTorch runs on CUDA, ROCm or Apple MPS.

5. **Verify the result.**
   Results go back through the gateway. The verifier sends a share of tasks to two or more workers and compares outputs, or recomputes a sample, before accepting them.

6. **Credit the user.**
   Each verified task adds credit to the ledger. Rejected or late results earn nothing and lower the worker's trust score.

7. **Pay out.**
   Once a balance passes the minimum, the payout service batches a transaction from the hot wallet to the user's address. It shows up in Cake Wallet after network confirmations.

Design constraints

GPU **PyTorch can't run inside a tab**

PyTorch is a native Python library and browsers can't load it or reach CUDA. The tab gets the GPU through WebGPU, so models are trained in PyTorch, exported to ONNX, and run with ONNX Runtime Web. Workloads that need full PyTorch (custom ops, training, large memory) need the optional local agent.

WALLET **Cake Wallet has no sign-up API**

Cake Wallet is self-custodial and open source, with no OAuth or account API for third parties. Blockhive can't create the wallet for the user. The user creates it in the app and gives you a public receive address. Never ask for the seed phrase or private keys.

AUTH **Keep secrets off the client**

Wallet keys, the hot wallet, and OAuth client secrets all live on the server. The current prototype puts a DCP private key in `App.js`; this design removes that.

TRUST **Treat every GPU as untrusted**

Users can fake results, so payment depends on verification. Run model code in a Web Worker with no network access beyond the job socket, and sign registry artifacts so the agent only runs approved models.

LIMITS **Respect the user's machine**

WebGPU caps buffer sizes and the browser throttles hidden tabs. Show a clear opt-in, a GPU usage cap, and pause work when the tab is hidden or on battery.